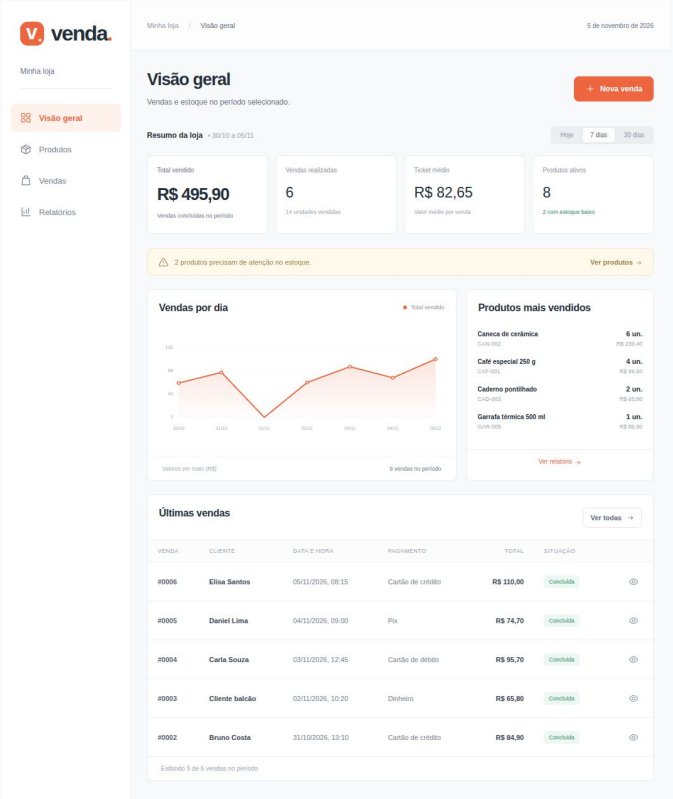
Sistema de controle de vendas em C

Projeto prático

Estácio S.A • Turma 3003

Professora: Suzane Alfaia Dias

Rodrigo de Sousa Barbosa | 202403618168
Manoel de Sousa Santos | 202402239521
Lucas Robert Lopes Silva | 201902231597
Alan Freitas Miranda | 202404621073
Gabriel Henrique Maciel Leite | 202403628988
Jorge Igor Dos Santos Pinto | 202309399989



Do cadastro ao resultado, um fluxo integrado.

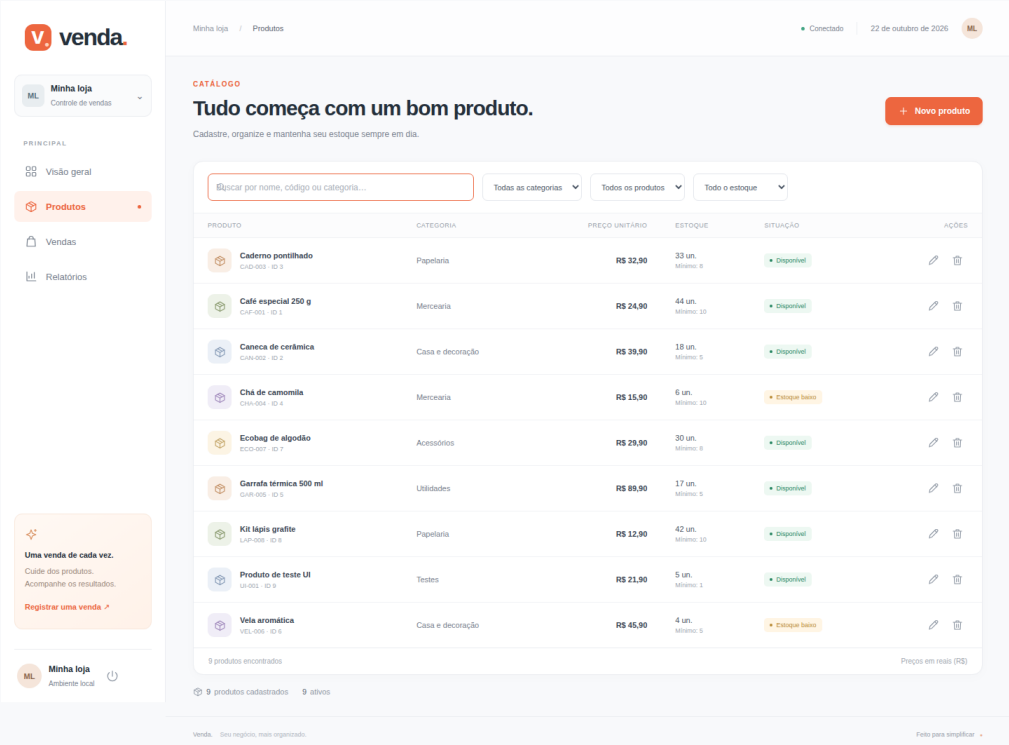
- Problema: registros dispersos, cálculo manual e estoque inconsistente.
- Público: responsável por uma pequena loja em uso local.
- Objetivo: cadastrar produtos, registrar vendas e saber o total vendido.
- Aplicar structs, funções, ponteiros, alocação e persistência em C.

Três camadas.
Uma única aplicação local.

- Interface gráfica: HTML, CSS e JavaScript, sem framework.
- Servidor C11/POSIX: HTTP, JSON, validações e regras de negócio.
- SQLite: produtos, vendas e itens persistidos em arquivo.
- Uma origem local: http://127.0.0.1:8080. Sem internet durante o uso.

Um catálogo com identidade e consistência.

- Product: id, sku, nome, categoria, preço, estoque, mínimo, situação e notas.
- Cadastrar, listar, pesquisar, atualizar e excluir.
- Código único e identificador interno automático.
- Produtos com vendas são desativados para preservar o histórico.



O total é calculado em C. O estoque acompanha.

- Subtotal = soma das quantidades × preços atuais do banco.
- Total = subtotal – desconto; valores inteiros em centavos.
- Produto ativo, quantidade positiva e estoque suficiente.
- BEGIN IMMEDIATE → validar → gravar → baixar → COMMIT.
- Falha em qualquer item: ROLLBACK da operação inteira.

Memória alocada para cada venda.

- O número de itens é conhecido somente em execução.
- Vetor dinâmico de SaleItem: calloc com verificação.
- Ponteiros acessam itens, requisições e comandos preparados.
- free em sucesso e falha; sem manter o vetor após a operação.

```
sale.items = calloc(sale.item_count,
sizeof(*sale.items));
if (!sale.items) { /* tratar falha */ }
/* ... processar itens ... */
free(sale.items);
```

Fechar o programa não apaga a loja.

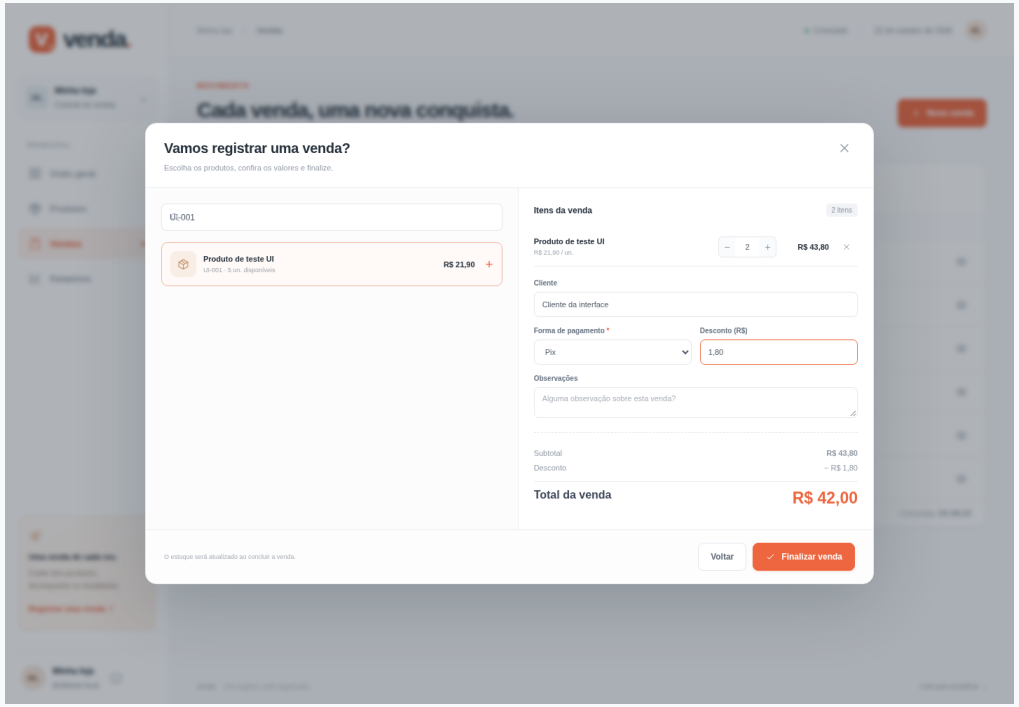
- Arquivo padrão: data/vendas.db.
- Tabelas relacionadas, CHECKs, SKU único e chaves estrangeiras.
- Nome, código e preço são copiados para cada item da venda.
- Demonstração: encerrar, iniciar novamente e consultar os mesmos registros.

Resultados executados, com evidências.

- 18 testes da API e do SQLite: aprovados.
- 9 fluxos no Chrome: aprovados.
- ASan/UBSan: mesmos 18 testes, sem erros detectados nos módulos próprios.
- Casos: vazio, código repetido, estoque, desconto, cancelamento e reinício.
- JSON, capturas e relatórios exportados em artifacts/.

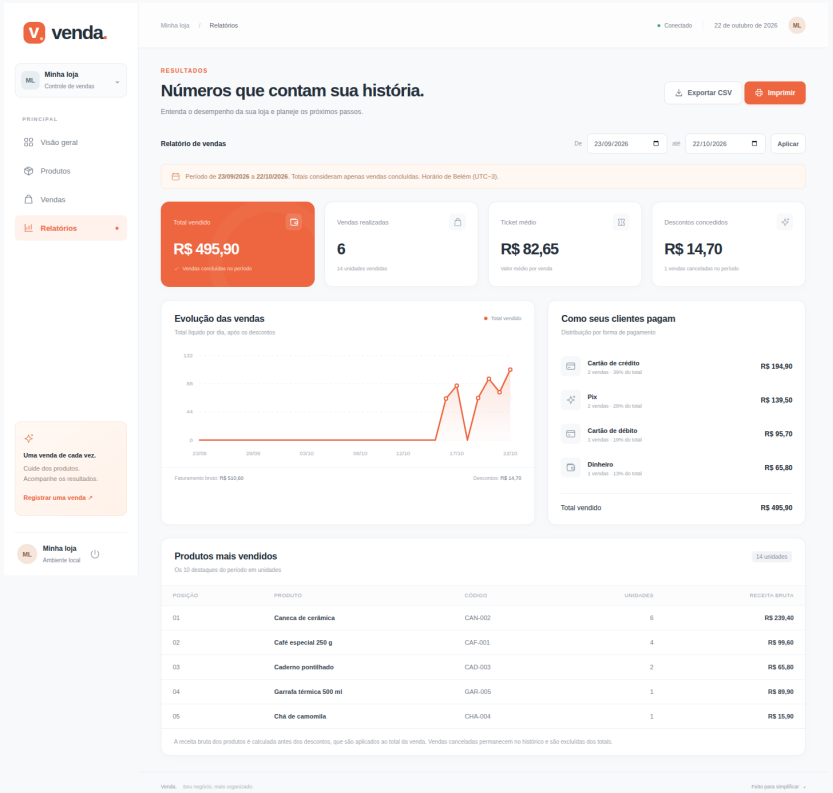
Uma venda, do início ao relatório.

1. Cadastrar um produto e pesquisar seu código.
2. Repetir o código: mostrar a rejeição e explicar a validação.
3. Adicionar itens, informar desconto e finalizar uma venda.
4. Conferir o comprovante e a quantidade restante.
5. Cancelar a venda: conferir a devolução única ao estoque.



O movimento da loja, em números claros.

- Período de datas no horário de Belém (UTC–3).
- Total líquido, vendas, ticket médio, descontos e unidades.
- Ranking por unidades e receita bruta dos produtos.
- Formas de pagamento, exportação CSV e impressão em PDF.
- Vendas canceladas ficam no histórico e não entram nos totais.



O essencial entregue.
Os próximos passos, claros.

- Entregue: C modular, CRUD gráfico, venda, relatório e persistência.
- Execução simples: make e ./iniciar.sh.
- Escopo: local, atendimento sequencial e controle interno.
- Melhorias futuras: usuários, operação remota e emissão fiscal.
- Toda a equipe deve compreender e explicar o código.

PERGUNTAS • 5 MINUTOS

Cada venda conta.
Cada decisão tem uma razão.

- Por que centavos inteiros?
- Como o programa evita uma venda parcial?
- Onde a memória é alocada e liberada?
- O que garante a recuperação após reiniciar?
- Obrigado.